



THREAT HUNTING WITH OPEN SOURCE TOOLS

Technical Report & Documentation

Intern Name: Arsalan Rasool Rather

Organization: CyArt

Program: Red Teaming Internship

Week: Week 2



1. Overview

This report documents the practical activities completed during Week 2 of the CyArt Red Teaming Internship Program. The focus of this week was Threat Hunting using open-source tools, specifically Elastic Security, Security Onion, and Sigma Rules. Key tasks included ingesting sample logs, writing a custom Sigma rule to detect suspicious PowerShell activity, and performing threat hunting queries inside Elastic Security.

2. Tools Used

- Elastic Security — for log ingestion and threat hunting queries
- Security Onion — for network security monitoring
- Sigma Rules — for writing vendor-agnostic detection rules

3. Practical Activity: Threat Hunting with Open-Source Tools

3.1 Lab Environment Setup

Before starting the hands-on tasks, the lab environment was configured. A Windows VM was used to simulate target activity, and Elastic Security was set up to receive and store process creation logs.

3.2 Log Ingestion into Elastic Security

Sample Windows process creation logs (Event ID 4688) were ingested into Elastic Security. The logs were collected from the Windows VM running the test PowerShell commands and forwarded to the Elastic stack for analysis.

3.3 Sigma Rule Creation — Detecting Suspicious PowerShell Activity

A custom Sigma rule was written to detect PowerShell command-line execution events. The rule targets the `process_creation` log source and filters for processes ending in `powershell.exe` that contain the `-Command` flag in their command line arguments.

Sigma Rule:

```
title: Suspicious PowerShell Activity
logsource:
  category: process_creation
```



```
product: windows
detection:
  selection:
    Image|endswith: '\\powershell.exe'
    CommandLine|contains: '-Command'
  condition: selection
```

The rule works by:

- Matching any process whose image path ends with powershell.exe
- Checking that the command line includes the -Command argument, which is commonly used in suspicious or malicious PowerShell execution
- Triggering an alert when both conditions are met simultaneously

3.4 Testing the Sigma Rule

The rule was tested using a harmless PowerShell command executed on the Windows VM to verify detection was working correctly. The command used for testing was:

```
powershell -Command "Write-Host Test"
```

This command safely triggers the rule conditions without causing any harm to the system. After execution, the event was captured by Elastic Security and matched against the Sigma rule.

3.5 Threat Hunting Query — Event ID 4688

A threat hunting query was run inside Elastic Security to search for all PowerShell-related process creation events identified by Windows Event ID 4688. The results were reviewed to identify potentially suspicious executions.

Query Results — Documented Events:

Timestamp	Process	Command Line	Notes
2025-08-18 10:00:00	powershell.exe	-Command Write-Host	Suspicious execution — matched Sigma rule
[Add from Elastic]			



4. Key Observations and Findings

- Sigma rules provide a vendor-neutral way to write detection logic that can be translated to multiple SIEM platforms including Elastic Security, Splunk, and Microsoft Sentinel.
- Event ID 4688 is a critical Windows event for process creation monitoring and is essential for detecting malicious PowerShell usage.
- The -Command parameter is frequently used in PowerShell-based attacks, making it a high-value indicator to monitor.
- Elastic Security's KQL (Kibana Query Language) enabled fast and flexible querying of large volumes of log data.
- Security Onion provided complementary network-level visibility alongside the host-level telemetry in Elastic.

5. Challenges Faced

- Initial log ingestion required configuring the Elastic Agent and ensuring Windows Event Forwarding was correctly set up on the VM.
- Sigma rule syntax required careful attention to indentation and field naming conventions specific to the Windows log schema.
- Differentiating between legitimate and suspicious PowerShell executions required understanding of the broader context beyond just the command line argument.
-

6. Learnings and Takeaways

This week provided valuable hands-on exposure to the threat hunting workflow used by professional red and blue teams. Key learnings include:

- How to write and structure Sigma detection rules for common attack techniques
- How to use Elastic Security for querying, filtering, and investigating security events
- The importance of process creation logging (Event ID 4688) as a foundational data source for threat detection
- How open-source tools like Elastic Security and Security Onion can be combined to build a capable threat detection environment

7. Next Steps

Based on Week 2 activities, the following areas will be explored in Week 3:

- Expanding Sigma rules to cover additional attack techniques such as lateral movement and credential dumping
- Correlating host-based events in Elastic Security with network events from Security Onion
- Exploring MITRE ATT&CK framework mappings for detected techniques